

Frontend FINANCE — l’application comptable

React 19 · TypeScript 5.9 · Vite 7 · Tailwind CSS 4 · port **3002** Dossier : frontend-finance/

Prérequis : le socle technique commun (monorepo, axios, Providers, thème, build) est décrit dans 02-FRONTEND-PROJECTS.md , sections 1 à 5.

SOMMAIRE

- 1. Une application volontairement plus légère
- 2. La garde d’accès globale
- 3. Le routage et la navigation
- 4. Le Dashboard financier
- 5. Module Factures & Devis
- 6. Module Autres revenus
- 7. Module Salaires — la paie tunisienne
- 8. Module Charges
- 9. Module Dettes
- 10. Module CNSS trimestriel
- 11. Module Paramètres de paie
- 12. La gestion des erreurs — un travail soigné
- 13. L’assistant IA Finance
- 14. Récapitulatif technique

1. Une application volontairement plus légère

Comparaison des dépendances

	frontend-projects	frontend-rh	frontend-finance
Nombre de dépendances	13	12	7
Pages	46	41	9
Lignes de code (pages)	~27 300	~24 000	~2 736
Ant Design	✓	✓	✗
Three.js (logo 3D)	✓	✓	✗
moment	✓	✓	✗
lucide-react	✓	✗	✗

Ses 7 dépendances :

```
@antigone/ai-chat-widget", "axios", "framer-motion",
"react", "react-dom", "react-icons", "react-router-dom", "recharts"
```

À dire en soutenance : « L’application Finance est dix fois plus petite que les deux autres. Ce n’est pas un manque de travail, c’est le résultat direct de l’architecture multi-frontend : un comptable n’a pas besoin de charger un moteur 3D, une librairie de composants complète et une librairie de dates. Chaque application n’embarque que ce dont son métier a besoin. »

Autre différence notable : pas de composants réutilisables lourds

L'application n'a que **trois composants UI** : - `AppSwitchButton` (basculer inter-applications, partagé) - `ErreurBanner` (bandeau d'erreur) - `Sidebar` / `MainLayout`

Les tableaux et formulaires sont écrits directement dans les pages. Pour 9 pages, c'est le bon compromis : abstraire aurait coûté plus qu'il n'aurait rapporté.

2. La garde d'accès globale

Toute l'application est protégée par une seule permission : `VIEW_FINANCE`.

```
/** Toute l'app Finance requiert VIEW_FINANCE – sinon on renvoie vers l'app RH. */
const FinanceGuard: React.FC<{ children: React.ReactNode }> = ({ children }) => {
  const { user } = useAuth();
  const allowed = user?.permissions?.includes('VIEW_FINANCE')
    || user?.roles?.includes('ADMIN');

  if (!allowed) {
    const rhAppUrl = import.meta.env.VITE_RH_APP_URL?.trim();
    return (
      <div className="min-h-screen flex flex-col items-center justify-center ...">
        <p>Accès refusé</p>
        <p>Votre compte ne dispose pas de la permission <strong>VIEW_FINANCE</strong>...</p>
        {rhAppUrl && <a href={rhAppUrl}>Retourner à l'espace RH</a>}
      </div>
    );
  }
  return <>{children}</>;
};
```

Appliqué **une seule fois**, autour du layout :

```
<ProtectedRoute>
  <FinanceGuard>
    <MainLayout />
  </FinanceGuard>
</ProtectedRoute>
```

Décision d'architecture à défendre : dans RH et Projects, chaque route porte sa permission (`PermissionRoute`). Ici, une seule garde suffit et couvre tout. C'est cohérent avec le métier : les données financières forment **un bloc indivisible**. Il n'y a pas de sens à laisser quelqu'un voir les charges sans voir les salaires.

Et côté serveur, la règle miroir existe dans `SecurityConfig` :

```
.requestMatchers("/api/finance/**").hasAnyAuthority("ROLE_ADMIN", "VIEW_FINANCE")
```

Message d'erreur soigné : au lieu d'une page blanche, l'utilisateur voit exactement quelle permission lui manque, avec un lien de retour vers l'espace RH.

3. Le routage et la navigation

Table de routage complète (9 routes)

/login	LoginPage	(20 lignes – redirige vers l'app Projects)
/dashboard	DashboardPage	Vue consolidée du mois
/factures	FacturesPage	Factures & devis
/revenus	RevenusPage	Revenus hors facturation
/salaires	SalairesPage	Bulletins de paie
/charges	ChargesPage	Charges fixes et variables
/dettes	DettesPage	Dettes de l'agence
/cnss	CnssPage	Déclarations trimestrielles
/parametres	ParametresPaiePage	Taux et barème IRPP

Une navigation à deux niveaux

`Sidebar.tsx` définit un **rail** de 6 icônes et des **panneaux** détaillés :

RAIL (icônes verticales)	PANNEAU au clic
--------------------------	-----------------

🏠 Tableau de bord	→	/dashboard
📄 Factures	→	· Factures & devis /factures
		· Autres revenus /revenus
☐ Salaires	→	· Salaires /salaires
		· Charges /charges
		· Dettes /dettes
📄 Charges	→	(regroupe charges + dettes)
🏠 CNSS	→	· CNSS trimestriel /cnss
		· Paramètres de paie /parametres
⚙️ Paramètres	→	/parametres

Le rail met en surbrillance la famille active grâce à des **préfixes d'URL** :

```
{ key: 'factures', prefixes: ['/factures', '/revenus'] }
{ key: 'charges', prefixes: ['/charges', '/dettes'] }
```

Ainsi, être sur `/revenus` allume bien l'icône « Factures ».

Le panneau « compte utilisateur »

Accessible depuis la barre, il permet de régler : - le **thème** (clair / sombre), - la **police d'écriture** (Inter / Poppins), - la **taille du texte** (13 / 14 / 15 / 16 px).

Ces réglages sont stockés en **cookie partagé**, donc conservés en passant à RH ou Projects.

Le bouton d'assistant relances

La barre latérale contient un bouton dédié :

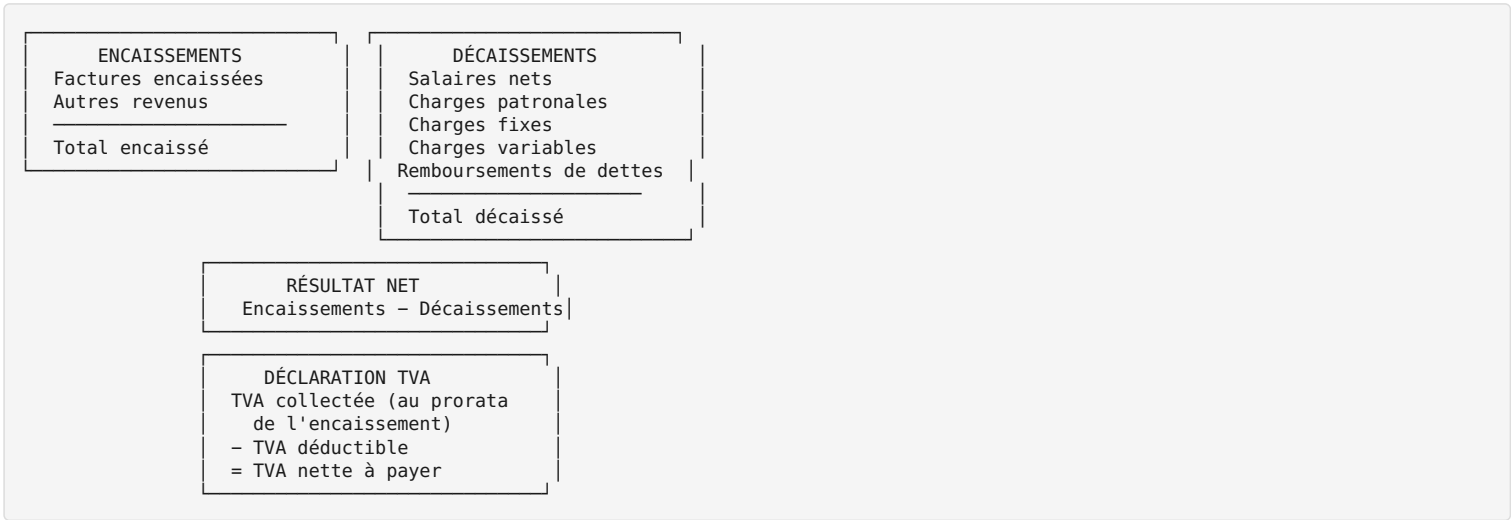
```
aria-label="Ouvrir l'assistant relances"
```

Il ouvre le widget IA avec une demande pré-remplie (voir §13).

4. Le Dashboard financier

DashboardPage.tsx (224 lignes) — la vue consolidée d'un mois.

Trois blocs



Le formatage monétaire tunisien

```
const fmt = (n) => (n ?? 0).toLocaleString('fr-TN', {
  minimumFractionDigits: 2, maximumFractionDigits: 2
});
// affichage : « 12 450,00 DT »
```

Détail de précision à remarquer : le dashboard formate à **2 décimales**, mais la page Salaires formate à **3 décimales**. Ce n'est pas une incohérence : le dinar tunisien se subdivise en **1 000 millimes**, et la paie se calcule au millime près (les taux CNSS/IRPP produisent des fractions). Les totaux financiers, eux, s'arrondissent au centime.

Navigation mensuelle

```
const currentMois = () => new Date().toISOString().slice(0, 7); // "2026-08"
```

Boutons < / > pour changer de mois. Tout le module Finance raisonne en **mois** YYYY-MM, exactement comme le backend.

5. Module Factures & Devis

FacturesPage.tsx — 629 lignes, la page la plus riche de l’application.

Un seul écran pour deux types de document

```
TypeDocument = FACTURE | DEVIS
```

Les règles métier appliquées par le backend et reflétées dans l’interface : - **Un devis n’a ni TVA ni timbre fiscal** (champs masqués) - **Le total HT saisi manuellement est un plancher**, pas un remplacement : `ht = max(somme des lignes, totalHtManuel)` - La numérotation est **séquentielle et sans trou** (`CompteurDocument`), obligation légale

Composition d’une facture

Bloc	Contenu
En-tête	Numéro (auto), type, client, date d’émission, date d’échéance
Lignes	Désignation, quantité, prix unitaire, ligne sélectionnée ou non
Totaux	Total HT, taux TVA (défaut du référentiel), montant TVA, timbre fiscal, Total TTC
Paielements	Plusieurs paiements partiels, montantPaye cumulé
Statut	EN_ATTENTE / PARTIELLEMENT_PAYÉE / PAYÉE / EN_RETARD
Relances	Historique des relances envoyées

Le lien avec l’IA

Une facture impayée peut être relancée par l’assistant : voir §13.

Endpoints consommés

```
GET/POST/PUT/DELETE /api/finance/facturation
                    /api/finance/parametres-facturation (taux TVA, timbre)
                    /api/finance/clients (liste des clients)
```

6. Module Autres revenus

RevenusPage.tsx (254 lignes) — les revenus **hors facturation** : subventions, prestations ponctuelles, remboursements.

Règle métier implémentée côté backend : > « *Le montant saisi est TTC, le HT en est extrait.* »

C’est l’inverse de la facturation (où l’on part du HT). Le comptable saisit ce qu’il a réellement reçu, et le système calcule la part de TVA — qui vient ensuite alimenter la **TVA collectée** de la déclaration mensuelle.

Chaque revenu porte : mois, libellé, montant, taux de TVA, date, catégorie (référentiel `CATEGORIE_REVENU`).

Endpoint : `/api/finance/revenus`

7. Module Salaires

SalairesPage.tsx (461 lignes) — le module le plus technique de l’application.

Le chargement initial

```
const [empRes, bullRes, totRes] = await Promise.all([
  referentielService.getEmployesDuMois(mois), // employés actifs ce mois-là
  payrollService.getBulletins(mois),         // bulletins déjà générés
  payrollService.getTotaux(mois),            // totaux consolidés
]);
```

`Promise.all` lance les **trois appels en parallèle** — un seul temps d’attente au lieu de trois.

`getEmployesDuMois(mois)` est important : le backend applique `PayrollCalculator.estActifPourMois()`, qui croise date d’embauche, date de fin de contrat et date d’archivage. Un employé arrivé le 20 du mois apparaît, un employé parti le mois précédent n’apparaît pas.

Les fonctionnalités de la page

Fonction	Description
Navigation mensuelle	< Août 2026 >
Génération des bulletins	POST /api/finance/paie/generer — calcule tous les bulletins du mois
Calcul unitaire	POST /api/finance/paie/calcul — simulation pour un employé
Détail d’un bulletin	Modale avec toutes les étapes du calcul
Acomptes	Enregistrement d’un acompte (montant + note) déduit du net à payer
Statut de paiement	IMPAYÉ / PARTIEL / PAYÉ (badge coloré)
Totaux	Masse salariale brute, nette, charges patronales, coût total

Les badges de statut

```
case 'PAYE':      return 'bg-success-50 text-success-600 ...';
case 'PARTIEL':  return 'bg-warning-50 text-warning-600 ...';
default:         return 'bg-error-50 text-error-600 ...'; // IMPAYE
```

Le détail d’un bulletin — ce que l’interface affiche

Le backend persiste **chaque étape intermédiaire**, ce qui permet d’afficher la chaîne complète (voir `01-BACKEND.md §12`) :

Salaire brut	2 000,000 DT
- Déduction absences	-83,333 DT
= Brut effectif	1 916,667 DT
- CNSS salarié (9,18 %)	-175,950 DT
= Salaire imposable	1 740,717 DT
- Abattement (10 %)	-174,072 DT
= Revenu net imposable	1 566,645 DT
- Contribution de solidarité (0,5 %)	-7,833 DT
- IRPP mensuel (barème progressif)	-243,120 DT
= Net	1 489,764 DT
- Acompte versé	-300,000 DT
= NET À PAYER	1 189,764 DT
Côté employeur :	
CNSS employeur · TFP · FOPROLOS · AT	
= Charges patronales → Coût total employeur	

Point clé de soutenance : « L’interface ne recalcule rien. Elle affiche des valeurs persistées en base. C’est ce qui garantit qu’un bulletin reste justifiable et identique même si les taux changent l’année suivante — et c’est aussi ce qui permet à l’IA de l’expliquer sans jamais recalculer. »

Endpoints

POST /api/finance/paie/generer	GET /api/finance/paie/parametres
POST /api/finance/paie/calcul	GET /api/finance/paie/baremes-irpp
POST /api/finance/paie/acompte	

8. Module Charges

`ChargesPage.tsx` (489 lignes) — deux catégories distinctes.

Charges fixes

Récurrentes : loyer, abonnements logiciels, assurances, internet. - Une charge fixe a un **montant mensuel** et des **paiements** (PaiementChargeFixe) - La page affiche l'**état de paiement** de chaque charge pour le mois courant (EtatChargeFixeDT0)

Charges variables

Ponctuelles : achats, déplacements, prestataires. - Montant, catégorie (référentiel CATEGORIE_CHARGE), date - **TVA déductible** — c'est elle qui vient en déduction dans la déclaration TVA

Endpoints : /api/finance/charges/fixes et /api/finance/charges/variables

9. Module Dettes

DettesPage.tsx (288 lignes) — les dettes de l'agence (emprunts, échéanciers fournisseurs).

Règle métier du backend, directement affichée :

```
/** Dettes de l'agence. soldeRestant = max(0, montantTotal - montantPaye). */
```

Chaque dette porte : libellé, créancier, montant total, montant payé, date d'échéance, et une liste de paiements (DettePaiement). Les dettes sont triées par **date d'échéance croissante** (findAllByOrderByDateEcheanceAsc) — la plus urgente en premier.

Endpoint : /api/finance/dettes

10. Module CNSS trimestriel

CnssPage.tsx (180 lignes) — la déclaration à la Caisse Nationale de Sécurité Sociale.

La suggestion automatique

Le backend somme les bulletins de paie des **3 mois du trimestre** :

```
montantSalarie += bulletin.getCnssSalarie() + bulletin.getSolidariteSalarie();
montantEmployeur += bulletin.getCnssEmployeurDetail();
```

L'échéance légale

```
/** Échéance légale de la déclaration : le 15 du mois suivant la fin du trimestre. */
public LocalDate echeanceTrimestre(int annee, int trimestre) {
    return LocalDate.of(annee, trimestre * 3, 15).plusMonths(1);
}
```

Exemple : T1 (janv-fév-mars) → échéance le **15 avril**.

La pénalité de retard

Calculée automatiquement si la déclaration est déposée après l'échéance, et ajoutée au montant total.

L'interface affiche par trimestre : montant salarié, montant employeur, pénalité, total, statut (payé / impayé), échéance.

Endpoint : /api/finance/cnss

11. Module Paramètres de paie

ParametresPaiePage.tsx (191 lignes) — la configuration fiscale et sociale.

Paramètre	Description
cnssSalarie	Taux de cotisation CNSS part salarié
cnssEmployeur	Taux part employeur
solidariteSalarie	Contribution sociale de solidarité

abattement	Taux d'abattement fiscal
tfp	Taxe de formation professionnelle
foprolos	Fonds de promotion du logement pour les salariés
at	Accidents du travail
Barème IRPP	Les 8 tranches progressives, éditables (plafond + taux)

Ce qui rend cette page importante : les taux tunisiens changent par loi de finances, quasiment chaque année. **Aucun taux n’est codé en dur dans le backend** : tout vient de `ParametresPaie` et `BaremeIrpp` . Un changement de loi = une saisie dans cette page, pas un redéploiement.

Le barème a un champ `effectiveFrom` : on peut **préparer le barème de l’année suivante** sans casser le calcul des bulletins passés.

Endpoints : `/api/finance/paie/parametres` et `/api/finance/paie/baremes-irpp`

12. La gestion des erreurs

`utils/apiError.ts` — **un fichier court mais remarquablement soigné**, à montrer au jury.

```
/**
 * Traduit une erreur axios en message lisible.
 *
 * Sans ça, un 403 ou un 404 se transforme en liste vide et l'écran affiche
 * « aucune donnée » alors que le problème est ailleurs.
 */
```

Le problème résolu

Sans ce module, un appel qui échoue produit `data = []` , et l'utilisateur voit « Aucune facture » — alors que le vrai problème est un backend éteint ou une permission manquante. **L’erreur devient invisible.**

La traduction, cas par cas

```
case 400: return 'Requête rejetée par le serveur (400) – exception côté backend, consultez ses logs.';
case 401: return 'Session expirée – reconnectez-vous.';
case 403: return 'Accès refusé : votre compte n'a pas la permission VIEW_FINANCE.';
case 404: return 'Endpoint introuvable – le backend n'expose pas encore cette route. Redémarrez-le.';
case 500: return 'Erreur serveur – consultez les logs du backend.';
```

Le cas CORS — le plus fin

```
// Un blocage CORS n'expose aucun statut HTTP au navigateur : il arrive ici,
// indistinguable d'un backend éteint. On cite donc les deux causes.
if (err?.code === 'ERR_NETWORK') {
  return "Requête bloquée : soit le backend est arrêté, soit l'origine de cette app "
    + "n'est pas autorisée par CORS (app.frontend-url doit contenir http://localhost:3002).";
}
```

Excellent point de soutenance : « *Un blocage CORS est invisible pour le JavaScript : le navigateur ne lui transmet aucun code HTTP. On ne peut donc pas le distinguer d’un serveur éteint. Plutôt que d’afficher “erreur réseau”, le message cite les deux causes possibles et donne directement la clé de configuration à corriger. C’est du diagnostic, pas juste un message d’erreur.* »

`ErreurBanner.tsx`

Un composant unique qui affiche le message produit. Toutes les pages suivent le même patron :

```
const [erreur, setErreur] = useState<string | null>(null);
...
catch (e) { setErreur(messageErreur(e)); }
...
{erreur && <ErreurBanner message={erreur} />}
```

13. L’assistant IA Finance

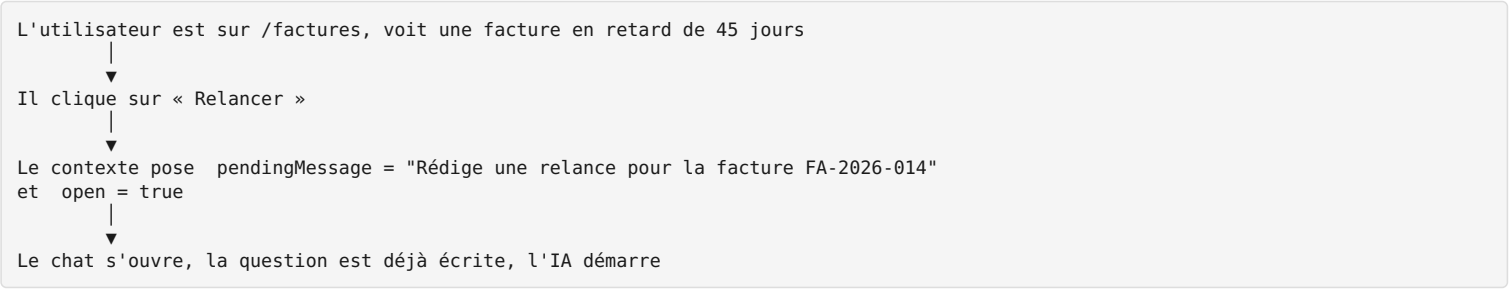
Un assistant global, spécialisé relances

```
<AiChatWidget
  apiUrl={API_BASE}
  getAuthToken={getAccessToken}
  onUnauthorized={handleUnauthorized}
  isAdmin={isAdmin}           // ADMIN ou VIEW_FINANCE
  open={assistant?.open}
  onOpenChange={assistant?.setOpen}
  initialMessage={assistant?.pendingMessage}
  theme={{ mode: theme, ... }}
/>
```

AiAssistantContext.tsx — l’ouverture pré-remplie

```
// Permet aux pages Finance d'ouvrir l'assistant avec une demande pré-remplie.
```

Le cas d’usage typique :



Ce que l’IA fait — et surtout ce qu’elle ne fait pas

- 1. InvoiceLookupTool → montants réels, jours de retard, palier de ton
- 2. Le PALIER EST IMPOSÉ par la règle métier, pas choisi par l'IA :
 - ≤ 7 jours → SOFT (rappel bienveillant)
 - ≤ 30 jours → FIRM (ferme mais courtois)
 - > 30 jours → FORMAL (formel, suites possibles)
- 3. EmailDraftTool → ENREGISTRE UN BROUILLON (n'envoie rien)
- 4. L'utilisateur LIT le brouillon
- 5. Il CLIQUE sur « Envoyer » → alors seulement l'email part

Le point de sécurité à défendre absolument : « L'IA ne peut pas envoyer un email de sa propre initiative. Elle rédige un brouillon, l'enregistre en base, et l'envoi exige un clic humain sur un texte lu. C’est une décision de conception explicite, écrite dans le code : “la validation humaine est le clic explicite de l'utilisateur sur le brouillon qu’il vient de lire”. »

Le widget affiche le brouillon dans un composant dédié (`ReminderResult.tsx`) avec l’objet, le corps, un aperçu HTML mis en page (logo du client inclus) et le bouton d’envoi.

Les autres capacités disponibles depuis Finance

Comme le backend classe l’intention de chaque message, l’assistant Finance sait aussi **expliquer un bulletin de paie** (capacité `PAYSLIP`) — utile quand un employé pose une question au service comptable.

14. Récapitulatif technique

Configuration

```
VITE_APP_KIND=finance
VITE_API_URL=http://localhost:8080
port dev : 3002
base de build : /Antigone_RH/finance/
```

Les 11 services API

authService · axios · chargesService · cnssService · dashboardService
detteService · employeService · facturationService · payrollService
referentielService · revenuService

Toutes les routes backend consommées

```
/api/auth/login · /forgot-password · /reset-password  
/api/finance/facturation  
/api/finance/parametres-facturation  
/api/finance/clients  
/api/finance/employes  
/api/finance/revenus  
/api/finance/charges/fixes  
/api/finance/charges/variables  
/api/finance/dettes  
/api/finance/cnss  
/api/finance/paie/generer  
/api/finance/paie/calcul  
/api/finance/paie/acompte  
/api/finance/paie/parametres  
/api/finance/paie/baremes-irpp  
/api/finance/dashboard
```

Les trois conventions transverses de l'application

1. Le mois est toujours au format YYYY-MM

```
const currentMois = () => new Date().toISOString().slice(0, 7);
```

Identique côté backend (`YearMonth.parse(mois)`). Aucune ambiguïté de fuseau horaire.

2. Le formatage monétaire est localisé `fr-TN` 2 décimales pour les vues financières, 3 décimales pour la paie (le dinar se divise en 1 000 millimes).

3. Toute page suit le même patron : `loading` → `Promise.all` des appels → `erreur` traduite par `messageErreur` → affichage via `ErreurBanner` .